

Oberta el dijous, 16 d'abril 2026, 09:09

Scalable and Elastic Ticket Service (AWS Managed Systems)

1. Objective

Design and implement a **distributed ticket-selling system** using managed AWS services.

Your system must:

- Ensure **correctness under concurrency**
- Implement **dynamic (elastic) scaling**
- Provide **reliable performance measurements**

2. Functional Requirements

You must support two ticket modes:

A. Unnumbered Tickets

- Maximum: 100,000 tickets
- No overselling allowed

B. Numbered Tickets

- Seats: 1–100,000
- Each seat can be sold **at most once**

👉 You must explicitly define:

- Your **consistency model** (strong vs eventual)
- How you prevent **race conditions**

3. Architecture Constraints

Your system must include:

✓ Asynchronous Processing (Mandatory)

Use a queue such as:

- RabbitMQ in a EC2 VM

This is required for:

- Load smoothing
- Decoupling components

✓ Stateless Workers

Use one of:

- AWS Lambda
- AWS Fargate

✓ Persistent Storage

- PostgreSQL in EC2 VM
- MySQL in EC2 VM

?

👉 You must justify:

- Consistency guarantees
- Transaction model

4. Realism Requirement

To simulate real-world systems:

- Each **buy transaction must include an artificial delay of 100 ms**
 - This models external payment processing latency
- The delay must be applied **inside the worker processing logic**

👉 This must be considered in:

- Throughput calculations
- Scaling decisions

5. Dynamic Scaling (Core Requirement)

You must implement scaling logic based on measured system load.

Suggested models:

Speedup

$$S = \frac{T_1}{T_N}$$

Where:

- S = Speedup
- T_1 = Execution time with a single worker (or single processor)
- T_N = Execution time with N workers (or processors)

Scaling based on arrival rate

$$N = \left\lceil \frac{\lambda}{C} \right\rceil$$

Where:

- N = Number of workers required
- λ = Arrival rate of messages (messages per second)
- C = Capacity of a single worker (messages per second)

Alternatively, if backlog is considered:

Scaling based on backlog

$$N = \left\lceil \frac{B + (\lambda \times T_r)}{C \times T_r} \right\rceil$$

Where:

- B = Current queue backlog (messages waiting to be processed)
- T_r = Target response time (seconds)

You must:

- estimate parameters experimentally
- map this to:
 - Lambda concurrency OR

ECS/Fargate scaling

6. Elastic Workload

You must design a **time-varying workload $Z(t)$** that demonstrates system elasticity.

The workload must include:

- Low load phase
- Gradual ramp-up
- Sudden spikes
- Sustained high load
- Cool-down phase

👉 Your system must:

- Scale up and down dynamically
- Avoid over-provisioning
- Maintain correctness during scaling

7. Stress Testing & Capacity Analysis

you must perform **stress tests** to determine:

- Maximum throughput per node
- Saturation point
- Performance degradation behavior

👉 You must:

- Gradually increase load until instability appears
- Identify:
 - Bottlenecks
 - Queue buildup
 - Latency growth

8. High Contention Scenario

You must evaluate:

A. Uniform Load

- Requests evenly distributed

B. Hotspot Load

- 80% of requests target 5% of seats

👉 This reveals:

- Lock contention
- Partition hotspots (especially in DynamoDB)

9. Throughput & Metrics Measurement (IMPORTANT)

A common mistake is measuring throughput incorrectly at the server.

✓ Required approach:

- Each **successfully processed request** must be recorded in:
 - A queue OR
 - A database/log structure
- You must track:
 - **Start time** of the experiment
 - **Completion time** (when all requests finish)

From this, compute:

- Throughput = completed requests / total time
- Latency distribution (p50, p95, p99)
- End-to-end processing time

👉 Important:

- Do NOT rely only on client-side timing
- Measurements must reflect **completed transactions only**

10. Fault Tolerance Requirements

You must implement:

- Worker failure
- Idempotency (request_id)
- Retry safety (at-least-once semantics)
- Dead-letter queues (SQS DLQ)
- Failure handling

11. Infrastructure as Code (Optional)

You must define your infrastructure using:

- **AWS CDK** (recommended)
OR
- **AWS CloudFormation** OR
- **Terraform**

👉 Requirement:

- One command deployment
- No manual setup

12. Optional Components

📬 Pub/Sub (Advanced)

- Amazon SNS
Only if:
 - Multi-stage pipelines OR
 - Separate analytics/logging

🚫 Not allowed as queue replacement

📦 Storage / Logging

- Amazon S3

📊 Monitoring

- Amazon CloudWatch
-

13. Deliverables

Code & Deployment

- Source code
- Deployment guide



Final Report (MANDATORY – STRICT REQUIREMENTS)

Your report must include:

Structure

- Table of Contents (TOC)
- Clearly organized sections

Architecture

- System architecture diagram(s)
- Component interaction explanation

Correctness

- Consistency model
- Concurrency control strategy

Performance Evaluation

- Experimental setup
- Workload definition (including $Z(t)$)
- Stress test methodology



Validation Plots (REQUIRED)

- Throughput vs workers
- Queue backlog vs time
- Latency percentiles



Plots must:

- Be clearly labeled
- Include interpretation (not just graphs)

Analysis

- Bottlenecks identified
- Scaling behavior explained
- Trade-offs discussed

AI Usage

- Explain how tools like pyrun.cloud, GitHub Copilot or others were used

14. Evaluation

Component	Weight
Correctness under concurrency	30%
Scaling implementation	35%
Architecture design	20%
Analysis & insights	15%

15. Conceptual Questions (To Be Answered in the Report)

Q1. Consistency vs Scalability Trade-off

In your system design:

- What consistency model did you choose and why?
- How would your system behavior change if you switched to a weaker/stronger model?
- What is the impact on correctness and performance?

Q2. Fault Tolerance vs Performance Trade-offs

- How do retries, idempotency, and failure handling affect performance?
- What overheads could fault tolerance introduce?
- Is there a point where increased reliability significantly reduces throughput?

Afegeix la tramesa

Estat de la tramesa

Número d'intent	Aquest és l'intent 1.
Estat de la tramesa	Encara no s'ha fet cap tramesa
Estat de la qualificació	No avaluada
Darrera modificació	-
Comentaris de la tramesa	> Comentaris (0)

◀ Task 1: Communication models and middleware

Salta a l'activitat

Introducció Python ▶